

✅ FINAL ARCHITECTURE (Medical Store Stock Manager)

🖥️ Architecture Type: 3-Tier Architecture

Frontend (GitHub Hosted Static Site)

↓ (HTTP via Tunnel URL)

Flask Backend (Localhost:5000)

↓

Supabase (Cloud PostgreSQL)

💠 1 FRONTEND LAYER (Presentation Layer)

Hosted on: GitHub Pages

Technology: HTML + CSS + JavaScript

Responsibilities

- Dashboard UI (like your reference image)
- Add Medicine Form
- Supplier Management
- Purchase Orders
- Sales Orders
- Stock Alerts
- Expiry Alerts
- Reports
- Authentication UI

Communication

`fetch("https://your-tunnel-url/api/products")`

Because:

- Backend runs locally
 - You expose it using ngrok / cloudflared tunnel
 - GitHub frontend calls tunnel URL
-

💠 2 BACKEND LAYER (Flask – API Layer)

Runs on: Localhost:5000

Exposed via: Tunnel

Responsibilities

- REST API
- Business logic
- Validation
- Stock calculation
- Expiry detection
- Authentication
- Role-based access (Admin / Staff)

Example Routes

POST /api/login

GET /api/dashboard-summary

GET /api/products

POST /api/products

PUT /api/products/<id>

DELETE /api/products/<id>

POST /api/purchase

POST /api/sale

GET /api/low-stock

GET /api/expiry-alerts

GET /api/reports/monthly

Flask acts as:

- Controller
- Service Layer
- Database connector

◆ 3 DATABASE LAYER (Supabase – PostgreSQL)

Cloud hosted database.

Backend connects using:

```
from supabase import create_client
```

Core Tables

1 users

column	type
--------	------

id	uuid
----	------

name	text
------	------

email	text
-------	------

password	text
----------	------

role	text (admin/staff)
------	--------------------

2 products

column	type
--------	------

id	uuid
----	------

name	text
------	------

category	text
----------	------

batch_no	text
----------	------

expiry_date	date
-------------	------

purchase_price	numeric
----------------	---------

selling_price	numeric
---------------	---------

quantity	integer
----------	---------

min_stock	integer
-----------	---------

created_at	timestamp
------------	-----------

3 suppliers

column	type
--------	------

id	uuid
----	------

name	text
------	------

phone	text
-------	------

column type

address text

purchases

column	type
--------	------

id	uuid
----	------

product_id	uuid
------------	------

supplier_id	uuid
-------------	------

quantity	integer
----------	---------

total_amount	numeric
--------------	---------

date	timestamp
------	-----------

sales

column	type
--------	------

id	uuid
----	------

product_id	uuid
------------	------

quantity	integer
----------	---------

total_amount	numeric
--------------	---------

date	timestamp
------	-----------

DATA FLOW (Example: Sale Process)

1. User clicks "Sell"

2. Frontend calls:

POST /api/sale

3. Flask:

- Checks stock
- Deducts quantity

- Inserts sale record
 - 4. Supabase updates
 - 5. Flask returns success
 - 6. Frontend updates dashboard
-

FINAL DEPLOYMENT ARCHITECTURE

User Browser



GitHub Pages (Frontend)



Tunnel URL (ngrok)



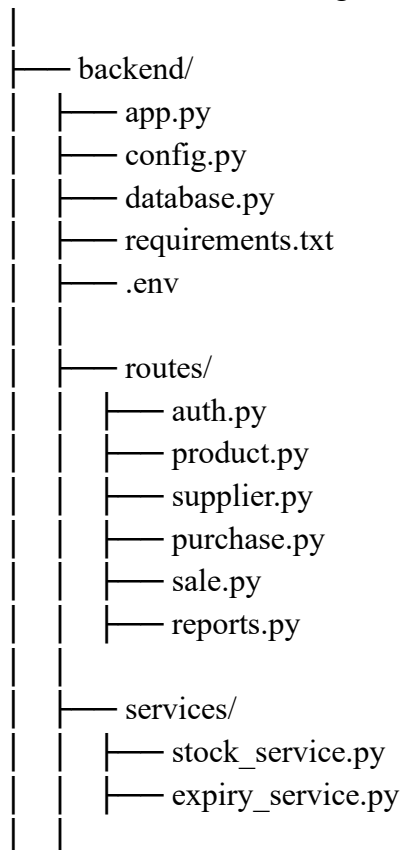
Localhost Flask Server

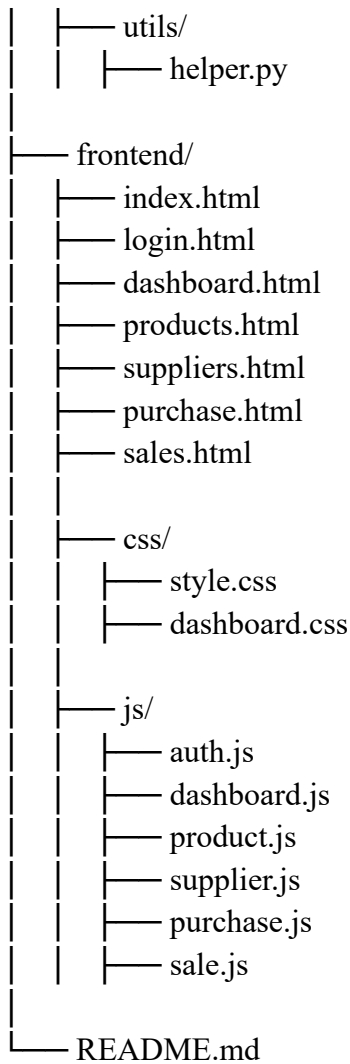


Supabase Cloud Database

FINAL FOLDER STRUCTURE

medical-store-stock-manager/





MASTER CONTROL PLAN (MCP)

This is your PROFESSIONAL execution roadmap.

PHASE 1 — Project Setup

Step 1 — Create Root Folder

medical-store-stock-manager

Step 2 — Create Backend

cd backend

python -m venv venv

Activate:

venv\Scripts\activate

Step 3 — Install Dependencies

`pip install flask flask-cors supabase python-dotenv`

Create:

`requirements.txt`

● PHASE 2 — Supabase Setup

1. Create Supabase project
2. Create tables (users, products, suppliers, purchases, sales)
3. Get:
 - `SUPABASE_URL`
 - `SUPABASE_KEY`

Create `.env`:

`SUPABASE_URL=xxxx`

`SUPABASE_KEY=xxxx`

● PHASE 3 — Backend Development

Order of Development

- 1 Database connection
- 2 Authentication
- 3 Product CRUD
- 4 Supplier CRUD
- 5 Purchase API
- 6 Sale API
- 7 Dashboard Summary API
- 8 Low stock API
- 9 Expiry API

Test using Postman.

● PHASE 4 — Frontend Development

Step 1 — Login Page

- Form

- fetch POST /api/login

Step 2 — Dashboard UI (Like Image)

Include:

- Total Products
- Low Stock
- Expired Items
- Total Value
- Graph (JS chart)
- Recent Purchases
- Top Selling
- Stock Alerts
- Expiry Alerts

Step 3 — Product Page

- Add Medicine
- Edit
- Delete

Step 4 — Connect Everything via fetch()

● PHASE 5 — Tunnel Setup

Install ngrok:

```
ngrok http 5000
```

You get:

```
https://abc123.ngrok-free.app
```

Update frontend API base URL:

```
const BASE_URL = "https://abc123.ngrok-free.app/api";
```

● PHASE 6 — Integration Testing

- ✓ Login works
- ✓ Add product

- ✓ Purchase updates stock
 - ✓ Sale deducts stock
 - ✓ Low stock auto detects
 - ✓ Expiry alerts work
 - ✓ Dashboard updates
-

● PHASE 7 — Error Handling & Security

- try/except
 - Proper status codes
 - CORS
 - Role based access
 - Password hashing (recommended)
-

🔒 SECURITY (Basic Version)

- Environment variables
 - Role-based control
 - CORS
 - Basic validation
-

🚀 FUTURE SCALABILITY PLAN

Later you can add:

- JWT Authentication
 - Barcode scanning
 - Invoice PDF generation
 - Online payment
 - Multi-branch support
 - Analytics dashboard
 - Excel export
-

HIGH LEVEL SYSTEM MODEL

Admin



Manage Products



Manage Suppliers



Purchase Stock



Sell Medicines



Track Reports